

PART II

Automatic Agent Evaluation (AutoEval)

Agent Safety · one grader, three runtimes; measure **where** it failed
and **whether** it was unsafe

Building Reliable Agent Evaluation

- **Thesis.** Outcome-only benchmarks reveal *that* an agent failed, not **where**, and not **whether it acted unsafely**. This work contributes a **repurposable grading pipeline [MINE]** that begins to address both.
- **Scope of contribution [MINE].** The **automated grading infrastructure**, off-device transcript reconstruction, LLM-judge customization, the Streamlit UI, and the task suite.
- **Attribution.** The **agent runtime and PEVA governance are the team's [TEAM]**; the **evaluation layer** is the contribution here.

Agent Evaluation: Why It Matters, Why It's Hard

WHY IT MATTERS

- **First: does it do the task correctly?** Today's agent eval, including this pipeline, is built for **accuracy verification**, and reliability is far from solved: human vs agent is **GAIA 92 vs 15**, **WebArena 78 vs 14** (SWE-bench, τ -bench climb fast; the gap holds).
- **Acting makes being wrong costlier.** A wrong tool call carries an **irreversible side effect** (email sent, money moved, data deleted). Side-effect verification is the **frontier**, barely built; this work names it and gives a taxonomy, not a solution.
- **So evaluation is the bottleneck, not capability.** As in PID, the missing piece is honest evaluation, not a cleverer model.

WHY IT'S HARD

- **Agents derail mid-task.** Wrong tool, collapsing reasoning chain, and you **can't localize which step failed**.
- **Outcomes aren't enough.** A lucky shortcut passes; good tool-use that ends wrong fails, neither is diagnosed.
- **Irreversible, and manual eval doesn't scale.** You can't reset between runs, and hand-grading every trajectory is neither reproducible nor scalable.

Current Evaluation Limitations (literature)

- **Outcome-dominant.** Across 9 surveyed benchmarks, most grade the **final state** (WebArena, Zhou et al., 2024; AndroidWorld, Rawles et al., 2025; GAIA; AgentBench). Per-step credit assignment is **rare**, only AgentBoard progress-rate and AgentBench per-step attempt it.
- **Side-effect verification** exists only in **heavy sandboxes / emulators** (AndroidWorld is an emulator); real-device verification is essentially absent.
- **LLM judges are widely used but under-audited.** ToolEval reports **87.1% pass agreement** with humans (Qin et al., 2023); ReasoningBank's self-judge is **72.7% vs ground truth** (Ouyang et al., 2025).
- **Agents remain far from reliable.** Human vs agent success: GAIA **92 vs 15**, WebArena **78 vs 14**, AndroidWorld **80 vs 31**.
- **Gap we target:** portable grading + the seed of **process-based** evaluation + honest judge auditing.

Related Work: How the Field Grades Agents

The 9 surveyed benchmarks span one axis, what counts as ground truth:

Ground truth	Method	Papers	Side effects?	Per-step?
structural	AST sub-tree match	Gorilla (Patil et al., 2023)	✗ (never executes)	✗
final answer	quasi-exact-match	GAIA (2023)	✗	✗
LLM-judge	ToolEval (≈87% human agr.)	ToolLLM (2023)	✗	weak
state / side-effect	DB/device inspection	WebArena, AndroidWorld, AgentBench	✓ (sandbox/emulator)	partial
per-step process	progress rate	AgentBoard, AgentBench	✗	yes (rare)

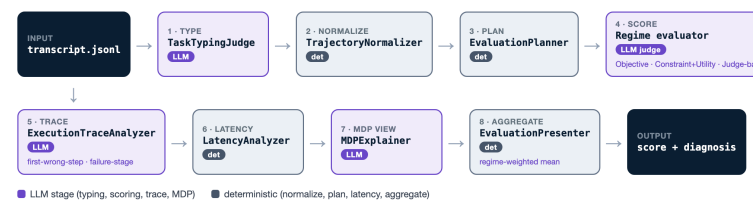
- **Two takeaways.** (1) The field moved from matching text → checking outcomes; **per-step "where did it fail" is still rare.** (2) LLM judges are ubiquitous but **under-audited** (ReasoningBank's judge ≈ 72.7% accurate).
- **This work's position:** a **portable** grader (transcript-as-contract), the **seed of process-based** evaluation, and an **audited** judge, exactly the field's open gaps.

Goal + Pipeline Overview

- **Goal. Automatic, reproducible** grading that returns both a **score** and a **diagnosis**.
- **The grader is a staged pipeline [MINE]**. Task-typing → trajectory normalization → a **regime evaluator** (objective / constraint+utility / judge-based) → execution-trace analysis → latency analysis → an MDP view → a weighted-score presenter. Deterministic checks and LLM-judge stages are kept separate.
- **Task-typed scoring**. A task's family selects a scoring regime with fixed weights (objective = outcome 0.70 / tool-use 0.30; hybrid = outcome 0.30 / constraint 0.30 / preference 0.20 / tool-use 0.20), so scoring is consistent rather than ad hoc.
- **Three runtimes, one grader**. OpenClaw (CLI) · a local mini-agent · KClaw on a real Galaxy S24, all emit the same transcript contract.

The Grader: a staged evaluation pipeline

source: `lib_evaluator.py` · entry point: `evaluate_episode()` · deterministic checks and LLM stages are kept separate



`scripts/lib_evaluator.py : evaluate_episode()`

Transcript-as-Contract

- **The key design.** Every runtime emits the **same JSONL event stream**: `{"type":"message","message":{"role, content[]}}, roles user / assistant / toolResult, blocks text | toolCall{name, arguments, id}`. The grader consumes only this contract, so it **never changes per runtime**.
- **Provider-agnostic by construction.** Anthropic `tool_use` blocks are translated to the canonical `toolCall` shape at the adapter boundary, so the judge and checks stay provider-independent.
- **Normalization.** `TrajectoryNormalizer` walks the stream into an `Episode` of steps, attaching each `toolResult` to its preceding call and recovering per-tool durations from the raw log.
- **Stress test [MINE].** The **OpenClaw** route was added *after* the pipeline was built around the local agent, **without touching the grader**. It passed, evidence the contract holds.

Reconstructing a Transcript Off a Real Phone

- **The problem.** KClaw runs on a physical S24; there is no clean transcript to grade.
- **The adapter (`lib_android.py`) [MINE].** All I/O over `adb shell`. **Inject** the prompt by writing a `user` row into the on-device **SQLite** `messages` table (`kclaw.db`) exactly as the UI does → **poll** that table every 3s for the bot reply (`is_bot_message=1`) → **read tool events** by `tail`-ing the on-device `agent.log` (`[ToolDispatcher]: Executing tool / Tool result / Tool completed`) → **merge both streams by timestamp** → emit the grader's JSONL.
- **Robust injection.** SQL is piped to `sqlite3` over stdin so the Android shell cannot mangle values (e.g. `$800` stays `$800`).
- **Honest limit.** Two **asynchronous** streams (DB messages, log tool events) are correlated **heuristically by timestamp**, and parsing is coupled to the `agent.log` line format `[VERIFY: confirm the current honest limitation]`.

LLM Judge: design + reliability

- **Input / output.** A summarized transcript + task rubric → a **single JSON** object `{"scores": {...}, "total": ..., "notes": ...}`, where `total` is constrained to the **arithmetic mean** of the criterion scores. Tool use is forbidden in the judge call.
- **Where it runs.** `claude-opus-4-5` (180s timeout) in the standalone grader, used for the **local and OpenClaw** modes. (*The KClaw device run was graded by deterministic autograders, not this judge.*)
- **Scoring discipline.** An early *strict* anchor ("reserve 1.0 for excellent") under-scored good work; the current design **starts at 1.0** and requires a **stated reason for every deduction** (see next slide).
- **Why so defensive.** LLM-as-judge carries known biases (position, verbosity, self-preference; Zheng et al., 2023) and returns unreliable JSON. Parsing is **3-pass**: `json.loads` → escape stray control characters and retry → **regex field extraction by key** (structure-immune); failures are dumped to a temp file for audit.

LLM Judge: Calibrating a Conservative Judge

- **The problem: LLM judges are conservative.** Under a plain "be strict" instruction the judge **under-scores work that was actually done well**, docking points it cannot justify.
- **The fix: force justified deductions.** The judge starts at **1.0**, treats a perfect score as correct when criteria are met, and must give a **specific reason for every deduction** (a required `deduction_reasons` field; per-step impacts sum to the shortfall).
- **The result, on a well-done task.** The trip-planning agent completed all components under budget. The calibrated judge scored **0.823**, deducting only **-0.15** (agent proposed **June 3**, a *reasoned* deviation outside the June 4–6 window) and **-0.05** (research-only tools), each with a stated reason, not a reflexive low score.

LLM Judge — a calibrated verdict: every deduction must be justified

task: `planning_01_business_travel` · model: `claude-sonnet-4-6` · the judge starts at 1.0 and may only deduct with a specific stated reason (fixes the conservative-scoring bias)



Result: End-to-End on a Real Phone, and the Honest Limits

- **The pipeline runs end-to-end on 49 real device tasks (physical S24) [MINE].** Transcript reconstruction → staged grading → per-step diagnosis, on **real hardware, not an emulator**. That the system runs on a live phone at all is the result.
- **Autograders handle the easy cases, and they are not enough.** Audited against human judgment they agree only **~74.5%** (35/47) [VERIFY] , and their false positives **miss real side effects**, they check the tool *call*, not whether the SMS actually fired.
- **That limit is the argument, not a footnote.** It is exactly why the **calibrated LLM judge** and **per-step diagnosis** exist, and why **real side-effect verification** is the next step. Don't trust a grader you haven't audited.
- **Attribution.** PEVA is the team's [TEAM]; the **pipeline, real-device reconstruction, and judge calibration** are [MINE].

Task Suite: 49 Real-Device Tasks by Side-Effect Tier

- Organized by the safety-relevant axis, side-effect tier, not by app: 22 read-only, 21 local-persistent, 6 external / irreversible. The 6 external tasks (SMS, email, Telegram, call, booking) are where a **user-confirmation gate** and real **side-effect verification** matter most.

KClaw Task Suite: 49 Real-Device Tasks by Side-Effect Tier

Physical Galaxy S24 · every task graded by deterministic autograders · tiers set the case for user-confirmation gates

None / read-only query or view, no state change → no gate needed	22	Local-persistent reversible on-device state → gate destructive ops	21	External / irreversible leaves the device, real-world effect → always gate	6
03 Web search		11 Create calendar event		02 Monitor Telegram + auto-reply	
04 Launch app		12 Edit calendar event		28 Send SMS	
05 Scroll page		13 Delete calendar event		33 Send Gmail	
06 Go to home screen		15 Save to memory		39 Send Telegram message	
07 Take screenshot		19 Create MiniApp		44 Make phone call	
08 Open a URL		20 Delete MiniApp		50 Reserve campsite (booking)	
09 Type in address bar		21 Create widget			
10 Open calendar		22 Delete widget			
14 Read notifications		24 Create note			
16 Search memory		25 Modify note			
17 List skills		26 Delete note			
18 Search skills		27 Schedule browser open			
23 Search files (PNG)		30 Create contact			
29 Find contact		31 Modify contact			
34 Battery status		32 Delete contact			
35 Storage info		37 Set alarm			
38 Screen time		38 Set timer			
40 Read clipboard		42 Take photo			
41 Navigate / directions		43 Toggle flashlight			
45 Calculator		47 Install app			
46 Check weather		49 Calendar + note (chained)			
48 Open app					

The side-effect tier, not the app, is the safety-relevant axis: the 6 external / irreversible tasks are where an autonomous agent needs a **user-confirmation gate**, and where side-effect verification (did the SMS actually fire?) matters most.

AutoEval Lessons + Common Theme

- **LLM judges fail in diagnosable ways**, audit them; measure agreement; engineer the context.
- **Beyond pass/fail**. The evaluator already flags a **first-wrong-step / failure-stage / root-cause**, the seed of **process-based** evaluation. **Next:** real side-effect verification.
- **The spine. PID prevents · AutoEval measures**, the basis for **trustworthy LLM systems**. Reliable evaluation infrastructure is the bridge from **model security** to **agent safety**.